



18 DE JUNHO DE 2025.

DEV PLATFORM
RELATÓRIO LOG 29

SÉRGIO PEREIRA
PEREIRA DEV
pereiradev.olution@gmail.com



Table of Contents

Análise de Engenharia do Relatório de Logs (Período de 01/06/2025 a 19/06/2025).....	2
Conclusão e Recomendações Estratégicas	5

Análise de Engenharia do Relatório de Logs (Período de 01/06/2025 a 19/06/2025)

Sumário Executivo

O relatório cobre um período de aproximadamente 18 dias, analisando 923 logs. A taxa de erro geral de 3.03% é relativamente baixa, o que é um indicador positivo da estabilidade geral. No entanto, a análise revela duas áreas principais que exigem atenção imediata:

1. **Gargalo de Desempenho Crítico:** A operação de listagem de usuários apresenta tempos de resposta inaceitáveis, chegando a mais de 4.6 segundos. Isso representa um risco significativo para a experiência do usuário e a escalabilidade do sistema.
2. **Sintomas de Fragilidade no Ciclo de Desenvolvimento:** Embora os 28 erros tenham sido resolvidos, sua natureza aponta para problemas sistêmicos no processo de refatoração, injeção de dependência e programação assíncrona.

A linha de tendência de volume de logs é ascendente, sugerindo um aumento na utilização do sistema. Esse crescimento torna a resolução do gargalo de desempenho ainda mais urgente.

1. Análise de Erros Resolvidos: Lições Aprendidas

Os 28 erros registrados, embora corrigidos, servem como um diagnóstico de pontos de atrito no nosso processo de desenvolvimento. Eles se enquadram em duas categorias principais:

a) Fragilidades de Refatoração e Injeção de Dependência:

- **Erros:**
 - 'SQLUnitOfWork' object has no attribute 'user_repository' (4 ocorrências).
 - 'SQLUnitOfWork' object has no attribute 'users' (2 ocorrências).
 - SQLUserRepository._init_() missing 1 required positional argument: 'logger' (2 ocorrências).
- **Diagnóstico:** Esses AttributeError e TypeError são sintomas clássicos de refatorações incompletas ou erros na configuração da injeção de dependência. É provável que o nome de um atributo na classe SQLUnitOfWork tenha sido alterado (users vs. user_repository), ou que a CompositionRoot não tenha fornecido a dependência ILogger ao instanciar o SQLUserRepository.
- **Recomendação Futura:** Para mitigar a reincidência, devemos fortalecer nosso pipeline de CI/CD com **testes de integração automatizados** que validem os

contratos entre as camadas após cada merge. A execução de testes que simulam os casos de uso de ponta a ponta teria capturado essas falhas de DI e refatoração antes do deploy.

b) Erros de Programação Assíncrona:

- **Erro:** cannot unpack non-iterable coroutine object (2 ocorrências).
 - **Diagnóstico:** Este é um erro comum em Python assíncrono, indicando que uma função de corrotina foi chamada sem o uso da palavra-chave `await`. O código tentou usar o objeto da corrotina em si, em vez de esperar pelo seu resultado.
 - **Recomendação Futura:** Aumentar o rigor da análise estática em nosso pipeline. Ferramentas como `mypy` e linters especializados em `asyncio` podem ser configuradas para sinalizar chamadas de corrotina não aguardadas, prevenindo essa classe de erro em tempo de desenvolvimento.
-

2. Análise de Desempenho: Ponto de Atenção Crítico

Este é o insight mais alarmante do relatório. As operações relacionadas à listagem de usuários são extremamente lentas:

- **Operações mais lentas:**
 - Users retrieved successfully: **4.62 segundos**.
 - Starting user listing: **4.53 segundos**.
 - Outras ocorrências com **3.47s** e **2.83s**.

Diagnóstico:

Um tempo de resposta de mais de 4 segundos para uma listagem de usuários é inaceitável e indica um gargalo severo. As causas mais prováveis são:

1. **Problema de N+1 Queries:** O código pode estar buscando a lista de usuários e, em seguida, executando uma nova query para cada usuário dentro de um loop para buscar dados relacionados.
2. **Ausência de Índices no Banco de Dados:** A tabela `users` pode não ter os índices adequados nas colunas usadas para filtragem ou ordenação.
3. **Transferência Excessiva de Dados:** A query pode estar selecionando colunas desnecessárias (`SELECT *`) ou trazendo um volume de registros muito grande de uma só vez, sem paginação.

Plano de Ação Sugerido:

1. **Revisão Imediata da Query:** Isolar o código do `ListUsersUseCase` e do `SQLUserRepository.find_all`. Analisar a query SQLAlchemy gerada (ativando `DB_ECHO=True` em ambiente de desenvolvimento).

2. **Análise do Plano de Execução da Query:** Executar um EXPLAIN na query diretamente no banco de dados para verificar se os índices estão sendo utilizados corretamente.
 3. **Implementar Paginação:** A função `find_all` deve ser modificada para aceitar parâmetros de `offset` e `limit`, evitando a busca de todos os usuários de uma só vez.
 4. **Adicionar Métricas de Desempenho:** Instrumentar o `ListUsersUseCase` com logs de tempo mais granulares para medir separadamente o tempo de acesso ao banco de dados e o tempo de processamento na aplicação (ex: mapeamento de DTOs).
-

3. Análise de Carga e Comportamento do Sistema

- **Volume de Logs:** O gráfico “Volume de Logs por Hora” mostra uma clara tendência de crescimento ao longo do período analisado. Isso indica um aumento na atividade do sistema, o que reforça a urgência em resolver os problemas de desempenho, pois eles serão exacerbados com o aumento da carga.
- **Picos de Atividade e Erros:** O relatório mostra picos de volume de logs significativos nos dias 14 e 17 de junho. O dia 14 de junho, em particular, teve um pico de **69 logs às 02:00** e coincide com a última ocorrência de vários dos erros listados¹⁶. Isso sugere que os picos estão correlacionados a deploys, testes de carga ou incidentes que foram posteriormente corrigidos.
- **Distribuição de Níveis de Log:**
 - INFO: 95.77%.
 - ERROR: 3.03%.
 - CRITICAL: 1.19%.
- **Diagnóstico:** A proporção é saudável, com uma baixa porcentagem de erros. No entanto, a presença de logs no nível CRITICAL (1.19%) merece atenção. Logs críticos devem significar uma falha grave que impede o funcionamento de uma funcionalidade central e deveriam, idealmente, ser extremamente raros e acionar alertas imediatos para a equipe de plantão.

Recomendação Futura:

- Revisar o código que gera logs CRITICAL para garantir que este nível está sendo usado apropriadamente.
 - Implementar um sistema de alertas automatizado para todos os logs de nível CRITICAL e ERROR para permitir uma resposta a incidentes mais rápida.
-

Conclusão e Recomendações Estratégicas

Embora o sistema demonstre boa resiliência com uma baixa taxa de erros, o relatório expõe pontos que precisam ser tratados proativamente.

1. **Prioridade Máxima: Investigar e corrigir o gargalo de desempenho na listagem de usuários.** Este é um problema que afeta diretamente o usuário final e a escalabilidade do sistema. O plano de ação descrito na seção de desempenho deve ser iniciado imediatamente.
2. **Prioridade Média: Fortalecer o pipeline de CI/CD.** Implementar testes de integração automatizados para validar a correta injeção de dependências e os contratos de interface entre as camadas. Aprimorar a análise estática para capturar erros comuns de programação assíncrona.
3. **Prioridade Baixa: Revisar a política de logging.** Assegurar que o nível CRITICAL seja reservado para falhas gravíssimas e que existam alertas configurados para os níveis ERROR e CRITICAL.